Testing Your Project

CSC490 Assignment 5

Janis Joplin (1009715051) • Louis Ryan Tan (1009713561)
Maverick Luke (1009714855) • Razan Ahsan Rifandi (1009562108)

January - April 2026

1 Profiling Execution time (10 marks)

1.1 ETL Profiling

Profiling revealed that the dominant cost in the ingest stage is Git subprocess I/O via `_run_git` and `subprocess.run`. However, several secondary Python-level costs were identified and optimized.

`_list_files` (Repository Traversal)

The ingest step walks the checked-out repository to enumerate files and collect metadata. On real-world repositories, this traversal can become expensive due to large dependency, build, and cache directories. To address this, the implementation was switched from a recursive glob-style approach to `os.walk`, with early pruning of common heavy directories such as `.git`, `node_modules`, and virtualenv and build folders via an ignore list. Output remains deterministic through sorting by relative path. Pruning avoids descending into directories that contribute many files but little analytical value, and `os.walk` reduces overhead by streaming results rather than constructing large intermediate lists. Where a repository consists mostly of source files with few ignorable directories, the improvement will be modest, and overall ingest time may still be dominated by Git I/O. To validate, `_list_files` timing should be compared across before-and-after ingest profiles using median-of-N runs, since filesystem caching makes single-run comparisons unreliable.

`_file_content_hash` (Content Hashing)

Hashing every file in a repository incurs disk I/O and CPU costs that are unnecessary for large binaries or asset files that carry no meaning for code-change analysis. The optimized implementation switches to `hashlib.file_digest(..., "sha256")` to reduce Python interpreter overhead, and skips hashing entirely for files exceeding a size threshold or matching common binary and asset extensions, returning an empty string to signal that no hash was computed. This avoids wasting time on irrelevant files while keeping hashing efficient for source files that matter. End-to-end ingest gains from this change are expected to be modest given that Git subprocess work remains the dominant cost; additionally, downstream consumers must not interpret an empty hash as indicating an unchanged file. Validation involves comparing the `_file_content_hash` contribution in before-and-after profiles and spot-checking that representative source files still receive non-empty hashes.

`extract_imports` (Dependency Graph Construction)

The transform stage builds a dependency graph by extracting imports from source files, and profiling identified `extract_imports` as a major hotspot due to repeated tree-sitter parsing and redundant per-file overhead. The optimized version in `src/pipeline/ast_imports.py` reads file bytes once and reuses them for both parsing and extraction, adds a cheap keyword pre-filter to skip tree-sitter parsing entirely when imports are clearly absent, and parses directly using the cached tree-sitter parser to avoid extra file I/O. This reduces wasted work in the dependency-graph step while preserving output behavior whenever parsing does occur. The keyword filter is heuristic and may miss cases where import-related keywords appear only in comments or strings, so not all unnecessary parses are eliminated. In practice, transform wall time on the Flask profile dropped by approximately 20% (from roughly 0.30s to 0.24s) after this change, which can be validated by profiling the transform stage before and after and comparing time spent in `extract_imports` as well as overall transform runtime.

1.2 RAG Profiling

From profiling, we see that the dominant cost in the indexing stage is tree-sitter parsing via `tree_sitter.Parser.parse`, accounting for roughly half of total wall time on code-heavy repositories. Several secondary Python-level costs were also identified. However, as indexing times on real-world repository examples remain acceptable for an MVP, the optimizations below are noted for future reference only and have not been implemented.

`_chunk_file` / `ts_base_strategy.chunk` (Tree-Sitter Parsing)

The profiling data across the two sample repositories shows `tree_sitter.Parser.parse` consuming approximately 0.113 s of 0.253 s total on the `click` repository (44 % of wall time), while remaining the dominant per-call cost on `markupsafe` as well. The cost scales with file count and is incurred even for files where AST-aware chunking yields little benefit over a simpler line-based split. The proposed optimization is to add a pre-filter in `_chunk_file` that bypasses tree-sitter strategies for very small files, binary-like files, and files with non-code extensions, falling back directly to the line-based strategy in `strategies.py`. This would reduce unnecessary parsing in mixed-content repositories such as those with many Markdown, YAML, or asset files. Expected impact is high for repositories with diverse file types, but modest for homogeneous Python or JavaScript projects where most files already benefit from AST-aware chunking. Validation should compare wall time in `_chunk_file` and `ts_base_strategy.chunk` before and after, using median-of-N runs to account for filesystem cache variance.

`_build_chunk_id` (Per-Chunk Hashing)

The profile for `click` shows `_build_chunk_id` called 1,619 times, driving 4,857 calls to `openssl_sha256`, indicating three hash operations per chunk. This represents a meaningful share of non-I/O CPU time at scale. The proposed optimization is to derive chunk IDs from a lightweight composite of `(repo_slug, file_path, start_line, end_line)` rather than hashing chunk content, reducing the hash count to one call per chunk. Alternatively, content hashing could be retained only for large chunks where collision avoidance is more important. Impact is medium; gains become more pronounced as corpus chunk counts grow. A key constraint is that downstream consumers must not interpret a position-based ID as proof of content identity, since two chunks at the same location across different repository states could share an ID despite differing content. Validation involves comparing `_build_chunk_id` and `openssl_sha256` call counts and cumulative time in before-and-after profiles.

`_build_requests` (Repeated Content Decoding)

The `_build_requests` method decodes `chunk.content` bytes for every chunk on every call, even when the decoded string has already been used for embedding in the same pipeline run. The proposed optimization is to store the decoded string either directly on the `Chunk` object or as a lazily-evaluated property, so that each chunk's bytes are decoded at most once. Impact is medium and grows with corpus size; for small repositories such as the profiled examples it is negligible. Validation involves confirming that decode call counts in the profiler drop proportionally with chunk count, and that embedding output is unchanged.

2 Showing Code Coverage (10 marks)

We decided to use `pytest-cov` (https://github.com/pytest-dev/pytest-cov) to measure the test coverage of the Python portion of our codebase, which in our case includes the ETL, pipeline, and backend modules under `src/` and `backend/`. We chose to build the automated Part 2 workflow around Python first because it allowed us to keep the testing and coverage pipeline within a single stack: one test runner, one coverage tool, one CI workflow, and one reporting format. This made the setup easier to reproduce, easier to debug, and easier to interpret consistently across modules.

We did not include the frontend in the dynamic workflow, although we still report its static coverage separately, because the frontend is written in JavaScript/TypeScript and would require a separate testing and coverage toolchain such as Jest or Vitest, along with a different reporting pipeline. Including both ecosystems in one automated setup would have added additional CI complexity and made the reported coverage less uniform across the repository. Since the main goal of Part 2 was to establish a clean and reliable automated coverage workflow, we prioritized a single-language end-to-end setup first, while still providing frontend coverage separately for completeness.

The full setup for this part can be seen in our pull request: https://github.com/UofT-CSC490-W2026/ContribNow/pull/13. The pipeline is as follows:

- We created a `pytest` configuration file, pytest.ini, to centralize the test setup and make it easy to run the full test suite with a single `pytest` command.
- In this configuration, we specified the test directory and enabled coverage reporting for the Python source folders, while also generating terminal, XML, and HTML coverage reports for both local inspection and CI integration.
- We then created a GitHub Actions workflow, test.yml, to automatically run the test suite on pull requests and on pushes to the relevant branch.
- Inside the workflow, GitHub Actions checks out the repository, sets up Python, installs the required dependencies from `requirements.txt`, and runs `pytest` with coverage enabled.
- The workflow uploads the generated HTML and XML coverage outputs as artifacts, so they can be downloaded and inspected directly from the Actions tab.
- For pull requests, we also integrated an automatic PR coverage comment, so reviewers can immediately see the number of passing tests, skipped tests, and overall coverage percentage after each update without needing to inspect the raw workflow logs or run the tests themselves.

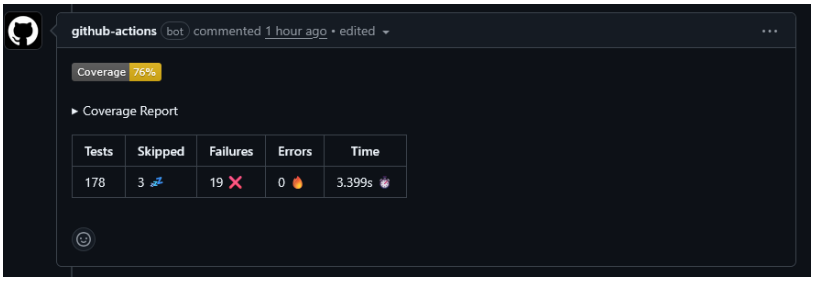
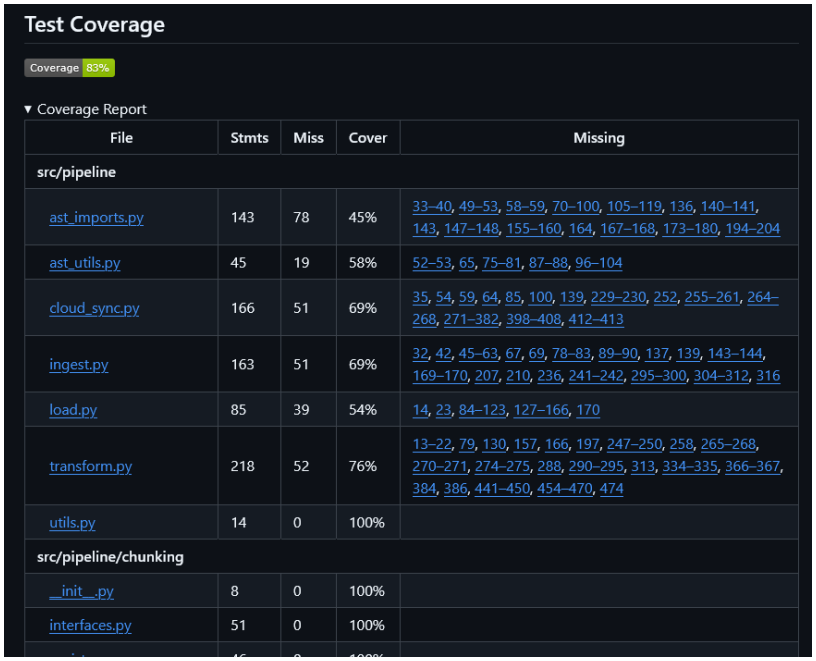


Figure 1: Automatic PR coverage comment generated by our GitHub Actions workflow. It summarizes the total coverage percentage together with the number of tests, skipped cases, failures, errors, and total runtime.

- We also integrated coverage into the root `README.md`, which can be viewed at README coverage section. This section shows the overall reported coverage, and the expandable `Coverage Report` dropdown provides a more detailed breakdown of the covered files.



Test Coverage

Coverage 83%

▾ Coverage Report

File	Stmts	Miss	Cover	Missing
src/pipeline				
ast_imports.py	143	78	45%	33–40, 49–53, 58–59, 70–100, 105–119, 136, 140–141, 143, 147–148, 155–160, 164, 167–168, 173–180, 194–204
ast_utils.py	45	19	58%	52–53, 65, 75–81, 87–88, 96–104
cloud_sync.py	166	51	69%	35, 54, 59, 64, 85, 100, 139, 229–230, 252, 255–261, 264–268, 271–382, 398–408, 412–413
ingest.py	163	51	69%	32, 42, 45–63, 67, 69, 78–83, 89–90, 137, 139, 143–144, 169–170, 207, 210, 236, 241–242, 295–300, 304–312, 316
load.py	85	39	54%	14, 23, 84–123, 127–166, 170
transform.py	218	52	76%	13–22, 79, 130, 157, 166, 197, 247–250, 258, 265–268, 270–271, 274–275, 288, 290–295, 313, 334–335, 366–367, 384, 386, 441–450, 454–470, 474
utils.py	14	0	100%	
src/pipeline/chunking				
__init__.py	8	0	100%	
interfaces.py	51	0	100%	

Figure 2: Coverage section integrated into the repository README. It displays the overall coverage percentage and provides an expandable file-by-file breakdown, so readers can inspect which modules are well covered and which still have missing lines.

To run the same test pipeline locally, one can simply run:

```
pytest
```

since the coverage options are already configured inside `pytest.ini`.

3 Test Coverage Breadth (20 marks)

Backend Test Files

322 tests pass with 3 skipped across 24 test files, achieving 100% coverage across all 1840 statements. The skipped tests are integration tests gated behind environment flags (`RUN_HF_TESTS=1`, `RUN_PG_VECTOR_TESTS=1`) that require large model downloads or a live Postgres instance, and are excluded from CI. Tables 1, 2, and 4 summarize each test file by subsystem.

Test File	Tests	What It Tests
`test_chunking_interfaces.py`	1	Validates that `ChunkingConfig` rejects invalid parameters such as zero `max_bytes`, overlap exceeding max, and `min_split` exceeding max. Prevents silent misconfiguration from corrupting chunks downstream.
`test_cloud_sync.py`	4	Tests payload redaction (stripping paths, emails, commit messages), deterministic fingerprinting, URL canonicalization, and `MemorySyncStore` idempotency. Ensures sensitive data never reaches the cloud layer and repeated syncs produce no duplicates.

Test File	Tests	What It Tests
`test_cloud_sync_extended.py`	23	Deep coverage of payload validation, file fingerprinting, `MemorySyncStore` and `PostgresSyncStore` with mocks, schema application, and the module-level `sync_cloud_safe` function including Postgres edge cases.
`test_embedding_batcher.py`	6	Tests batch splitting by size, max bytes/tokens enforcement, total token budget splits, and missing token counter validation. Guards the embedding pipeline against exceeding provider limits.
`test_huggingface_provider.py`	1 (skipped)	Integration test for `BAAI/bge-code-v1` model download and embedding generation; gated behind `RUN_HF_TESTS=1` to avoid large model downloads in CI.
`test_indexing_in_memory.py`	3	End-to-end in-memory indexing (ingest JSON → chunk → embed → upsert) and `delete_by_repo` cleanup. Proves the full indexing pipeline produces searchable vectors.
`test_indexing_failures.py`	12	Failure scenarios: missing `head_commit`, empty hashes, file read errors, chunking errors, embedding provider errors, vector store errors, and validation failures. Ensures the indexer degrades gracefully under every failure mode.
`test_language_registry.py`	6	Language detection by extension, special filenames (`Dockerfile`, `Makefile`), shebang lines, unknown files, strategy registration, and global singleton behaviour. Ensures files route to the correct chunking strategy.
`test_naive_chunking.py`	3	Naive byte-based chunking: empty content, overlap between consecutive chunks, and last chunk ending at content boundary. Validates the fallback strategy when no tree-sitter parser is available.
`test_pipeline_etl.py`	10	Full ETL integration: ingest schema v2, transform gold-layer features (co-change, risk, authorship, dependency graph, conventions), load artifact indexing, `.env` parsing, and failure isolation. Validates the entire pipeline from git clone to output artifact.
`test_ts_code_chunking.py`	4	Tree-sitter semantic chunking for JavaScript, Java, and JSX using fake AST nodes; tests language mismatch rejection. Ensures code splits at meaningful boundaries rather than arbitrary byte offsets.
`test_ts_py_chunking.py`	3	Tree-sitter Python chunking with fake AST, parser build failure handling, and non-Python language rejection. Validates Python-specific semantic chunking and graceful failure when tree-sitter is unavailable.
`test_ast_imports.py`	35	AST-based import extraction for Python, JavaScript, and Java; relative import resolution; dependency graph building; and extraction edge cases. Ensures the dependency graph accurately reflects actual code imports.
`test_ast_utils_extended.py`	14	Tree-sitter parser infrastructure: language detection, parser caching, graceful degradation when tree-sitter is unavailable, and the `parse_file` utility. Validates the shared parser layer all chunking strategies depend on.

Test File	Tests	What It Tests
`test_ingest_unit.py`	30	Unit tests for git operations (clone, fetch, branch detection), slug generation, file listing, commit log building, and the main ingest entry point. Isolates ingest logic from real git repos.
`test_load_unit.py`	11	Environment file loading, transform artifact loading, cloud sync integration, and the main load entry point. Ensures the load stage writes artifacts and updates the index without data loss.
`test_transform_deep.py`	33	Structure summary building, hotspot computation, co-change matrix with `MAX_FILES_PER_COMMIT` guard, authorship tracking, risk scoring, 11+ convention pattern matchers, and dependency graph computation. Isolates every transform sub-computation.
`test_postgres_vector_store.py`	10 (1 skipped)	PgVectorStore schema creation, upsert with conflict handling, search with filtering, and live round-trip integration; gated behind `RUN_PG_VECTOR_TESTS=1`.

Table 1: Pipeline test files (`tests/`) and their coverage scope.

Test File	Tests	What It Tests
`test_chunking.py`	27	Comprehensive chunking tests: config validation, language registry, naive strategy, tree-sitter semantic chunking for Python/JS/Java/JSX, and edge cases. End-to-end validation of the entire chunking subsystem.
`test_embedding.py`	27	Batch request logic, OpenAI provider, HuggingFace provider, `LocalEmbeddingProvider`, token counting, and `max_tokens`/`max_bytes` constraints. Covers all embedding providers and validates batching limits.
`test_indexing.py`	13	In-memory vector store operations, file iteration with hashing and filtering, chunk file processing, embedding request building, and integration. Validates the RAG indexing pipeline from file discovery through vector upsert.
`test_vector_store.py`	27 (1 skipped)	`InMemoryVectorStore` cosine similarity, `PgVectorStore` with PostgreSQL, upsert, search with filtering, delete, and live round-trip integration. Ensures both vector store backends return correct nearest-neighbour results.

Table 2: RAG test files (`tests/rag/`) and their coverage scope.

Test File	Tests	What It Tests
`test_config_models_and_services.py`	14	Backend config loading, Pydantic models, and service layer (auth, cache, DB, LLM, prompt builder, retrieval, storage) with mock DB connections. Validates all backend services in isolation.
`test_main.py`	8	FastAPI lifespan, root endpoint, debug endpoint, and the full onboarding generation flow including caching and error handling. Ensures the API layer correctly orchestrates services and returns proper HTTP responses.

Table 4: Backend API test files (`tests/backend/`) and their coverage scope.

Frontend Test Files

All 69 tests across 10 test files pass with 100% coverage across statements, branches, functions, and lines. Table 5 summarizes each test file and what it covers.

Test File	What It Tests
`App.test.tsx`	Top-level integration: child component rendering, conditional loading/error/result display, form submission, and regeneration flow. Verifies that state flows correctly between hooks and UI across full user workflows.
`client.test.ts`	`generateOnboarding`, `healthCheck`, and `ApiError`: request construction, error parsing, mock mode, and abort signal propagation. Guards the network boundary against malformed requests and untyped errors.
`AccessKeyInput.test.tsx`	Input rendering, value display, password/text visibility toggle, and `onChange` callback. Ensures credentials are masked by default, the toggle works, and changes propagate correctly.
`ErrorDisplay.test.tsx`	User-facing error messages for 401, 500, other 5xx, network errors, and unknown status codes. Ensures users see clear, actionable messages rather than raw HTTP codes or stack traces.
`Header.test.tsx`	Rendering of the app name, beta badge, and tagline. Acts as a smoke test to catch accidental removal of key branding elements.
`LoadingIndicator.test.tsx`	Spinner rendering and progressive status messages that advance on a timer and stop at the final message. Validates that users receive feedback during long API calls.
`OnboardingDocument.test.tsx`	Markdown rendering (including GFM tables), cached/fresh badges, version display, and regenerate button visibility and callback. Protects the core output component from regressions.
`RepoForm.test.tsx`	URL validation, access key enforcement, loading/disabled states, custom prompt toggle, and form submission with and without a custom prompt. Guards the primary input flow against bad submissions and missing data.
`useAccessKey.test.ts`	`localStorage` read/write for the access key: initial load, update, and overwrite. Ensures key persistence across sessions so users do not re-enter their key on every visit.
`useGenerateOnboarding.test.ts`	Async state machine: loading, success, `ApiError`, network error, `AbortError` suppression, reset, and request cancellation. Validates state transitions, request deduplication via abort, and clean resets.

Table 5: Frontend test files and their coverage scope.

4 Test Coverage Depth (20 marks)

We chose the indexing module (src/pipeline/indexing) as the most complex because it connects several subsystems into one workflow. It chunks files, creates embeddings from them, and stores it into a vector store. Therefore, it has core logic that must succeed and produce correct results, and failure modes that must be handled correctly. Any failure in any of the stages may influence other stages, resulting in unexpected behaviour. Furthermore, some

failures are recoverable while others may be fatal. For example, treesitter-based chunking might fail if the file content is severely malformed, but this failure can (and should) be recovered using a fallback naive chunking strategy. Meanwhile, a database failure or failure on creating embeddings are not necessarily recoverable. Because of these considerations, we have created comprehensive unit tests for this module, covering both success and failure paths, to ensure predictable behaviour.
We have created more than 10 unit tests for this module. For brevity, we have chosen 10 tests that we considered most important. The 10 tests can be grouped into 3 sections:

4.1 Success Path

This test whether the system works correctly when everything is valid. This is important so that we are able to trust that the failure tests we are doing are meaningful. We created one test that confirms the entire pipeline runs end-to-end and produces the correct output.

Test Name	**Test Path**	**Why It Matters**
Valid indexing with preserved metadata	`test_indexing_in_memory.py ::TestIndexingInMemor :: test_indexes_and_searches`	This is the main success path. It verifies that a valid file is indexed, stored, and later searchable, while preserving `head_commit` metadata.

4.2 Input Validation

These tests targets input-handling logic which runs before the module runs chunking or embeddings. These are important so that the module rejects bad or suspicious inputs instead of trying to run them and potentially failing.

Test Name	**Test Path**	**Why It Matters**
Missing `head_commit` in the ingest manifest	`test_indexing_failures.py ::TestIndexingFailures :: test_missing_head_commit_raises`	Ensures indexing cannot proceed without revision identity, required for correct later reconstruction.
Empty-hash file entries skipped by default	`test_indexing_failures.py ::TestIndexingFailures :: test_skips_empty_hashes_by_default`	Checks that unstable or incomplete file metadata is excluded by default.
Empty-hash file entries included when configured	`test_indexing_failures.py ::TestIndexingFailures :: test_can_include_empty_hashes_when_disabled`	Verifies the skip behaviour is configurable rather than hard-coded.
Path escape attempts in legacy manifests are rejected	`test_indexing_failures.py ::TestIndexingFailures :: test_legacy_manifest_skips_paths_outside_repo`	Protects the indexer from reading files outside the repository root.

4.3 Failure Propagation

Finally, we created a set of tests for failures that happen during pipeline execution. These tests make sure that the system can handle failures mid-operation gracefully.

Test Name	Test Path	Why It Matters
Unreadable files skipped without aborting the run	`test_indexing_failures.py ::TestIndexingFailures :: test_read_errors_skip_only_the_unreadable_file`	Confirms the pipeline continues indexing when one file cannot be read.
Empty chunk output does not trigger embedding or upsert	`test_indexing_failures.py ::TestIndexingFailures :: test_empty_chunk_output_does_not_send_embeddings`	Validates that empty chunk sets do not cause meaningless downstream work.
Chunking failures propagate	`test_indexing_failures.py ::TestIndexingFailures :: test_chunking_errors_propagate`	Ensures chunking-stage corruption or runtime failures are not silently ignored.
Embedding provider failures propagate	`test_indexing_failures.py ::TestIndexingFailures :: test_embedding_provider_errors_propagate`	Confirms embedding-stage failures fail loudly instead of producing partial or misleading results.
Vector store upsert failures propagate	`test_indexing_failures.py ::TestIndexingFailures :: test_vector_store_errors_propagate`	Verifies persistence failures are treated as hard errors, critical for indexing correctness.

5 Two Memorable Bugs (40 marks)

Part 5 video walkthrough: https://drive.google.com/drive/folders/1Jwue8y5ZPTe3yTHakYyVnjzgk-m3X-rx?usp=sharing This video covers two memorable bugs, including how we discovered them, debugged them, fixed them, and the regression tests added afterward.

References